

Full Stack Web Development FS2025

Arbeitsauftrag

Im Kurs gestaltet und entwickelt ihr dieses Jahr ein Collaboration-Tool in Python mit Flask in Gruppen zu je 2-3 Studierenden. Den Funktionsumfang legt ihr gemeinsam im Team fest. Der Kreativität sind allerdings keine Grenzen gesetzt. Auch Mobile-Features wie der Einbezug der Position (Geo-Location) sollen nicht zu kurz kommen.

Im Kurs geht es darum, mit Copilot eine App zu entwickeln. Ihr sollt lernen und sehen, wie viel leichter es mit Copilot ist, aber wo auch die Grenzen sind. Wir erwarten jedoch, dass ihr die Grundlagen aus Software-Engineering 1 und 2 sowie aus Data-Management noch zur Verfügung habt.

Wir geben dabei eine Einführung in die Zusammenarbeit mit Github und die Verwendung von Copilot im Visual Studio Code Editor.

Im Kurs folgt ihr tageweise einem strikten Plan mit jeweils gesetzten, minimalen Tageszielen.

Gruppeneinteilung und Rollen in der Gruppe

Wir sind uns bewusst, dass es Unterschiede in den Programmierkompetenzen gibt und auch nicht jeder alle Tätigkeiten gleich gerne ausführt. Es ist aber auch so, dass man mit Copilot immer noch ein weiteres Teammitglied hat, das programmieren kann. Daher bieten wir euch eine abwechslungsreiche Gruppenarbeit an. Damit das gut funktioniert, müssen wir sicherstellen, dass die Gruppen etwa gleichverteilte Kompetenzprofile aufweisen.

Dazu füllt ihr im Vorfeld einen kurzen Fragebogen auf Moodle aus. Wir übernehmen dann die Einteilung nach «best effort».

In den Gruppen soll es folgende Rollen und Profile geben:

	Technisches Profil	Gestalterisches Profil
Frontend	X	
Backend	X	
Datenbank / Datenmodell	X	
User Experience		X
Requirements-Engineering		X
Test-Engineering	X	

Zeitplan und Tagesziele

Grundfunktionalitäten

- View: Benutzerprofile erfassen (mit Foto), Teams bilden, Chatfunktion im Team, gemeinsame Dateiablage
- Search: Parametrische Suche nach Personen und deren Skills (z.B. Programmierkenntnisse, Project Management, etc.)
- Vorschlag: Simpler Match-Algorithmus für Menschen mit ähnlichen Skills
- Chat: 1:n Senden und Empfangen von Text-Nachrichten.

(Mögliche) Sonderfunktionalitäten + eigene Ideen

Geo-Location / Distanz, KI-Antworthilfe, Projektplanungstool, Telefonie oder Videocall, Email / SMS / Whatsapp-Notification zu Nachrichten und Profilbesuchen, usw.

Nice-to-Have

Online-Status / Gesendet-Gelesen von Nachrichten / «Vergessen» von Nachrichten nach X Tagen

Tagesablauf:

Wann?	Wo?	Thema
08:30 - 09:00	SI O2.02	Check-In, Anwesenheit, Fragen, Tagesinformationen
09:00 - 12:00	SI O1.12, SI O1.13, SI O4.12, SI O4.13	Gruppenarbeit
12:00 - 13:00	SW-Mensa	Mittagessen
13:00 - 16:30	SI O1.12, SI O1.13, SI O4.12, SI O4.13	Gruppenarbeit
16:30 - 17:00	SI O2.02	Check-Out, Reflektion, kurzer Report zum Stand (1 min / Gruppe).

Tag 1:

	Ziel
Frontend	Login und Anzeige des persönlichen Profils
Backend	Kompletter Durchstich via Controller/Entities zur Datenbank
Datenbank / Datenmodell	ER-Modell mit 10 Dummy-Daten.
User Experience	Anforderungserhebung durch gegenseitige Interviews. 3 Personas / Szenarien (Problem und Aktivität) Papier-Prototyp zu den Grundfunktionalitäten (Search / View / Chat / Match)
Requirements-Engineering	
Test-Engineering	

Tag 2:

	Ziel
Frontend	Teambildung / Search / View / Chat implementiert
Backend	Komplette Abbildung der Grundfunktionen und Controller/Entities zur Datenbank
Datenbank / Datenmodell	Vollständiges ER-Modell der Grundfunktionalitäten.
User Experience	Erweiterung der Anforderungen, der Szenarien und des Designs in Richtung Sonderfunktionalitäten und Nice-to-Have.
Requirements-Engineering	
Test-Engineering	

Tag 3:

	Ziel
Frontend	Grund- und Sonderfunktionalitäten so weit wie möglich implementiert und bereit für einen Testlauf.
Backend	
Datenbank / Datenmodell	
User Experience	Entwicklung von Test-Prozeduren, die am Tagesende ausgeführt werden. Tagesziel ist hierbei eine priorisierte Bug-Liste, die in den Folgetagen abgearbeitet werden soll.
Requirements-Engineering	
Test-Engineering	

Hack-Werkstatt (SI O2.02)

Anlaufpunkt bei allen (technischen) Problemen. First-Come-First-Served, durchgehend besetzt. Donnerstag/Freitag per Teams für 2h/Tag.

Bewertung / Erwartung

Die Bewertung erfolgt anhand der final abgegebenen Projekte (Link auf einen Git-Release + Video-Demo). In der Video-Demo stellt ihr euren Prototypen gemeinsam vor. Dabei soll auch das ganze Projekt mit allen zusätzlichen Artefakten (Szenarien, Test-Prozeduren) reflektiert werden. Das Video darf max. 10 Minuten dauern und jeder Teilnehmer muss zu Wort kommen und einen sinnvollen Teil des Projekts präsentieren. Zusätzlich haltet ihr in einem Projekt-Tagebuch fest, wer wie viele Stunden an welchem Teil gearbeitet hat.

Abgabe via Moodle (alles mit Timestamp vor dem 06.07.2025 / 23:59h)

- Link zum GitHub-Release – duda/heej sind als Contributors freigeschaltet.
- Link zum Video-Beitrag via Teams aufgenommen (max. 10 min!)
- Tagebuch als Excel/PDF (Template auf Moodle verfügbar)

Benotung als grundsätzliche Richtgrösse:

1. Wer ausschliesslich die Grundfunktionalitäten implementiert bekommt und diese angemessen funktionieren, kann den Kurs mit einer 4.0 bestehen. Voraussetzung

ist aber auch hierbei, dass alle drei Artefakte in vernünftiger Qualität abgegeben werden.

2. Richtung 6.0 geht es mit Quantität UND/ODER Qualität von Zusatzfunktionen und exzellentem UX-Design. Bewertet wird Front- und Backend.